

Object Mapping

Object Mapping

`gemstone-js` supports object mapping, but it keeps the `GemStone` object model explicit. The library maps JavaScript values and TypeScript types to `GemStone` calls and object handles; it does not automatically hydrate `GemStone` objects into mutable JavaScript class instances.

That distinction is deliberate. `GemStone` objects have identity, session affinity, transaction visibility, and remote selector-send semantics. Hiding those behind plain JavaScript property reads would make common workflows look simpler while making correctness harder to reason about.

Mapping Layers

Use these layers according to how much object identity you need to preserve:

- **Value marshalling:** `performWith()`, `performValueWith()`, `ManagedOop.send()`, arrays, and dictionaries convert common JavaScript values into `GemStone` objects and marshal simple results back.
- **Raw identity:** `Oop` is the branded `bigint` handle for a `GemStone` object. Use raw OOP APIs when you need exact identity or want to avoid result marshalling.
- **Retained typed handles:** `TypedOop<T>` keeps a `GemStone` object retained for the session and gives TypeScript a compile-time witness for the expected domain type.
- **Class references:** `Session.classRef<T>()` caches class lookup and exposes class-side sends, allocation, and object-returning sends.
- **Generated wrappers:** `codegen` manifests and decorated source emit typed functions that call `GemStone` selectors and return values, raw OOPs, or retained typed handles.
- **Dictionary payloads:** `objectToDictionaryArgument()` converts plain objects or class instances into explicit `StringKeyValueDictionary` payloads before persistence or selector sends.
- **Value converters:** `ValueConverterRegistry` lets sessions marshal richer scalar values such as `Date` without changing the default persistence contract.

Typed Object Handles

Use `TypedOop<T>` when a `GemStone` selector returns a live object and you want TypeScript to remember the expected domain shape.

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

const session = await Session.connect();
const booking: TypedOop<Booking> = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const status = await booking.send<string>("status");
await booking.send("status:", "confirmed");
```

```
await session.commit();
await booking.release();
```

This is object mapping through a retained remote handle. The `Booking` type is a compile-time witness for the handle; it is not a hydrated local instance. Selectors still make remote calls, and writes still follow the active GemStone transaction.

Generated Selector Wrappers

Generated wrappers are the most ergonomic mapping layer for application code. The manifest or decorator source declares the GemStone class, selector, argument types, and return kind. The generated function keeps the selector send explicit while giving callers a typed API.

```
{
  "functions": [
    {
      "exportedName": "findBooking",
      "className": "Booking",
      "selector": "find:",
      "argNames": ["id"],
      "argTypes": ["string"],
      "returnType": "TypedOop<Booking>",
      "returnKind": "object"
    }
  ]
}
```

The generated output calls `classRef().sendObject()`:

```
export async function findBooking(
  session: Session,
  id: string,
): Promise<TypedOop<Booking>> {
  return session.classRef("Booking").sendObject("find:", id);
}
```

Use `returnKind: "value"` for simple marshalled values, `returnKind: "oop"` for raw object identity, and `returnKind: "object"` for retained typed handles.

Explicit Dictionary Payloads

When you want to persist or pass a JavaScript object as structured data, convert it explicitly into a dictionary argument.

```
import {
  Session,
  objectToDictionaryArgument,
  scalarValueConverterRegistry,
} from "gemstone-js";

class BookingDraft {
  constructor(
    public id: string,
    public status: string,
    public requestedAt: Date,
  ) {}
}

const session = await Session.connect({
  valueConverters: scalarValueConverterRegistry(),
```

```
});

const draft = new BookingDraft("B-1001", "held", new Date("2026-05-19T12:00:00Z"));
const payload = objectToDictionaryArgument(draft);

const booking = await session
  .classRef<Booking>("Booking")
  .sendObject("createFromDictionary:", payload);
```

This mirrors the explicit `dataclass_to_dict()` style used in `gemstone-py`. Class instances become dictionary payloads only when the caller opts in.

Dictionary Mapping

`GemStone StringKeyValueDictionary` is the most direct bridge between JavaScript records and GemStone object storage. Use it when the shape is keyed, reviewable, and not worth modeling as a full GemStone class.

```
import { Session, type TypedOop } from "gemstone-js";

interface Booking {
  id: string;
  status: string;
}

const session = await Session.connect();
const booking = await session
  .classRef<Booking>("Booking")
  .sendObject("find:", "B-1001");

const dict = await session.dictionary({
  status: "held",
  tags: ["vip", "late-arrival"],
  limits: { guests: 2, bags: 3 },
});

await dict.setObject("booking", booking);
await dict.setAllValue({
  owner: "front-desk",
  priority: 3,
});

const status = await dict.requireValue("status");
const bookingAgain: TypedOop<Booking> = await dict.requireObject<Booking>("booking");
const limits = await dict.requireDict("limits");
const entries = await dict.items({ maxEntries: 50 });
const rawEntries = await dict.itemsOop({ maxEntries: 50 });
```

The value/object/raw naming is intentional:

- `getValue()/requireValue()` marshal the stored GemStone value back into a JavaScript value when possible.
- `getObject<T>()/requireObject<T>()` return retained `TypedOop<T>` handles for live GemStone objects.
- `getOop()/requireOop()` and `itemsOop()` preserve raw object identity.
- `getDict()/requireDict()` wrap nested dictionaries as `GsDict`.

When you only have a raw dictionary OOP, the session-level helpers expose the same mapping operations without creating a long-lived wrapper:

```
const dictOop = await session.dictionaryToOop({
  status: "held",
```

```
    updatedBy: "api",
  });
```

```
await session.dictionarySetObject(dictOop, "booking", booking);
const values = await session.dictionaryEntries(dictOop, { maxEntries: 100 });
const handles = await session.dictionaryEntriesOop(dictOop, { maxEntries: 100 });
const mappedBooking = await session.dictionaryRequireObject<Booking>(dictOop, "booking");
```

For durable application state, dictionaries usually live under a persistent root or global. The root helper keeps the same mapping choices visible:

```
import { PersistentRoot } from "gemstone-js";

const root = PersistentRoot.userGlobals(session);
const index = await root.setDict("BookingIndex", {
  kind: "booking-index",
  active: true,
});

await index.setObject("B-1001", booking);
await root.setObject("LastBooking", booking);

const savedIndex = await root.requireDict("BookingIndex");
const savedBooking = await savedIndex.requireObject<Booking>("B-1001");
const rootBooking = await root.requireObject<Booking>("LastBooking");
```

Dictionaries are a good mapping target for configuration, indexes, metadata, API payload snapshots, and small keyed aggregates. Prefer class references and typed object handles when the GemStone object has behavior that should remain on the GemStone side.

Value Converters

Converters run before built-in argument marshallng. They are best for scalar domain values that should cross the GemStone boundary in a consistent form.

```
import { Session, scalarValueConverterRegistry } from "gemstone-js";

const session = await Session.connect({
  valueConverters: scalarValueConverterRegistry(),
});

await session.classRef("AuditLog").send("recordAt:", new Date());
```

The built-in scalar registry stores `Date` values as ISO strings. Custom converters can add application-specific scalar wrappers while keeping object graph persistence explicit.

Readback and Inspection

For simple data structures, use bounded value readback:

```
const values = await session.arrayOopToValues(arrayOop, {
  maxDepth: 4,
  maxTotalItems: 500,
});

const dict = await session.dictionaryOopToObject(dictOop, {
  maxEntries: 100,
});
```

For live GemStone objects, prefer retained handles and inspection:

```
const object = session.typedOop<Booking>(bookingOop);
const summary = await object.inspect();
const dump = await object.dump({ maxDepth: 2, maxItems: 50 });
```

Bounded readback protects callers from unexpectedly large collections or deep object graphs. Inspection keeps object identity visible instead of pretending the remote object is a plain local value.

Mapping Manifest Roadmap

The next object-mapping step should be connector-inspired, but still explicit and async. The goal is to keep GemStone identity visible while giving application code a smaller, typed surface than raw selector sends.

1. Mapping manifest schema

Add a dedicated mapping manifest alongside the existing function-codegen manifest. Each mapped class should declare the GemStone class name, the TypeScript domain type, selectors, setters, repository selectors, and snapshot fields.

```
{
  "$schema": "./schemas/object-mapping-manifest.schema.json",
  "classes": [
    {
      "name": "Booking",
      "gemStoneClass": "Booking",
      "typeName": "Booking",
      "refName": "BookingRef",
      "repository": {
        "className": "BookingRepository",
        "find": {
          "name": "find",
          "selector": "find:",
          "args": [{ "name": "id", "type": "string" }]
        }
      },
      "selectors": [
        { "name": "status", "selector": "status", "returnType": "string" }
      ],
      "setters": [
        { "name": "setStatus", "selector": "status:", "args": [{ "name": "status", "type": "string" }]
      ],
      "snapshot": [
        { "name": "id", "selector": "id", "type": "string" },
        { "name": "status", "selector": "status", "type": "string" }
      ]
    }
  ]
}
```

2. Generated *Ref classes

The generator should emit small reference classes that wrap TypedOop<T>. Methods remain async and selector-backed; they do not become JavaScript properties.

```
export class BookingRef {
  constructor(readonly object: TypedOop<Booking>) {}

  get session(): Session {
    return this.object.session;
  }
}
```

```

get oop(): Oop {
    return this.object.oop;
}

status(): Promise<string> {
    return this.object.send<string>("status");
}

async setStatus(status: string): Promise<this> {
    await this.object.send("status:", status);
    return this;
}

async snapshot(): Promise<BookingSnapshot> {
    return {
        id: await this.object.send<string>("id"),
        status: await this.status(),
    };
}

inspect() {
    return this.object.inspect();
}

release() {
    return this.object.release();
}
}

```

3. Repository helpers

Repository helpers should return typed refs rather than raw `TypedOop<T>` handles. They can wrap class-side selectors, persistent-root lookups, query helpers, or `GemStone` repository objects.

```

export class BookingRepository {
    constructor(readonly session: Session) {}

    async find(id: string): Promise<BookingRef> {
        const object = await this.session
            .classRef<Booking>("BookingRepository")
            .sendObject("find:", id);
        return new BookingRef(object);
    }
}

```

4. Snapshot and dictionary helpers

Generated refs should support bounded `snapshot()` methods for UI/API payloads. Snapshot output should be plain data, not retained `GemStone` handles. Dictionary-backed snapshots should use the existing `dictionaryOopToObject()`, `GsDict`, and `PersistentRoot` helpers with `maxEntries` bounds.

5. Explorer and VS Code mapping views

The Explorer and VS Code workbench can later read mapping manifests to show mapped classes, generated ref methods, repository helpers, and snapshot fields. This should be a tooling view over committed generated source, not a hidden runtime mapper.

This roadmap intentionally differs from a transparent JavaScript ORM. It is closer to the useful parts of the Pharo bridge connector model: reviewable class pair metadata, generated accessors, repository entry points, and inspectable mapping state. It avoids automatic local/`GemStone` synchronization until the typed ref and snapshot

path is proven against live Stone workflows.

What Is Not Automatic

`gemstone-js` does not currently provide:

- transparent `booking.status` property dispatch to GemStone selectors
- automatic JS class hydration from arbitrary GemStone instances
- a session-wide identity map for local wrapper instances
- automatic persistence of arbitrary JavaScript object graphs
- lazy slot loading hidden behind synchronous property access
- bidirectional local/GemStone synchronization through connector rows

Those features can be useful in narrow domains, but they also create sharp edges around transactions, remote latency, conflict handling, and object identity. The current object-mapping API keeps those boundaries reviewable.